

# When the Golden Model Is Wrong Too: Industrial-Grade Verification for an Open-Source GPGPU

Samuel Moussa\*, Steven Ibrahim\*, Ahmad Sudky\*,  
Ahmad Fawzy\*, Abanoub Nabil\*, Alhassan Sayed\*,  
Hossam Hassan<sup>†</sup>, and Hyung-Min Yoon<sup>‡</sup>

\*Department of Electronics and Communications Engineering,  
Faculty of Engineering, Minia University, Egypt

<sup>†</sup>Independent Researcher, South Korea    <sup>‡</sup>Siliconarts, Inc., South Korea

Corresponding author: samuelmoussa64@gmail.com

**Abstract**—Open-source RISC-V GPGPUs ship with directed-kernel regressions and little else—no reference-model checking, no functional-coverage model, no sign-off discipline—even as security research shows real defects populate exactly this class of design. We built the missing half for Vortex, the leading open RISC-V GPGPU: a complete UVM 1.2 environment with per-instruction, per-lane lockstep against the project’s own functional simulator, a two-pass technique for verifying racy multi-core programs, and the first SIMT functional-coverage model published for RTL GPU verification. The experience delivered a catalog of lessons D&T readers will recognize: assertions silenced to make a bench run stay silenced for months; fixing a bug can make your coverage number go *down*; a coverage definition can itself be defective; and—the headline—a lockstep flow audits the golden model as rigorously as the RTL, catching reference bugs that a fuzzing campaign had mislabeled as hardware defects. We share the methodology, the numbers, and the failure stories, with every claim carrying its evidence.

**Index Terms**—UVM, RISC-V, GPGPU, SIMT, lockstep verification, functional coverage, open-source hardware

## I. THE BUG THAT WAS HIDDEN BY A SILENCED ASSERTION

A library counter in Vortex—the open-source RISC-V GPGPU [1]—ships with assertions that check for overflow and underflow. During verification bring-up, they fired. The quick fix, standard in every lab that has lived through bring-up: guard the assertions to skip whenever their inputs are unknown, so the environment can run. The guard stayed. For months.

When we finally restored the original assertions and asked *why* they had fired, the answer was not a testbench nuisance. Reset in this design is distributed through a relay that registers the incoming reset in a flip-flop nothing initializes; for one cycle, every module behind that relay sees an unknown reset, `if (X)` evaluates false, and reset-held logic executes its normal-operation paths on unknown operands. The assertions had been correct the entire time. What we had suppressed was the report, not the problem.

The fix—an asynchronous-assert, synchronous-deassert synchronizer—made twelve assertion firings vanish with the unmodified upstream file. It also did something coverage people rarely see: *branch coverage went down*, from 95.1%

to 94.5%. Modules behind the relay had been executing normal-operation paths during an illegitimate state, and those executions had been counted as covered branches. No coverage metric can tell you that some of its coverage came from a state that cannot legitimately arise. We report the lower number.

This article is the story of a verification methodology built to have such stories—and of what it found when aimed at both sides of a comparison: a RISC-V GPGPU and its own golden model.

## II. THE GAP: SIGN-OFF-GRADE VERIFICATION FOR OPEN GPGPUS

The RISC-V world solved reference-model verification for *scalar* cores years ago. Open-source step-and-compare environments [3], constrained-random instruction generators [4], and standard verification interfaces [5], [6] are table stakes; industrial open-silicon projects run UVM flows at SoC scale [7], and agile CPU projects industrialized per-instruction co-simulation over DPI [8].

The GPGPU—the most complex open processor class—had none of it. Vortex shipped a directed-kernel regression, CI, and nothing you could call a verification methodology. Meanwhile FuzzGPU [2] (USENIX Security 2026) demonstrated that a fuzzer targeting the same design class finds real bugs: twenty previously unknown defects and ten CVEs across Vortex and Ventus. That result converts the missing methodology from an academic inconvenience into an exposure.

Fuzzing and sign-off are complementary halves of one problem, and this work is the other half. A fuzzer’s job is to find bugs fast; it measures itself in findings, runs on structural coverage, and stops when the campaign budget ends. A sign-off methodology’s job is to quantify how thoroughly the design has been exercised; it measures itself in functional coverage closed under auditable exclusions, with checkers proven able to fail. The two even cross-validate: as we will see, they found the same ISA deviation independently, and we later ran *their* reported bugs through *our* environment, with a result that vindicated our methodology’s central thesis.

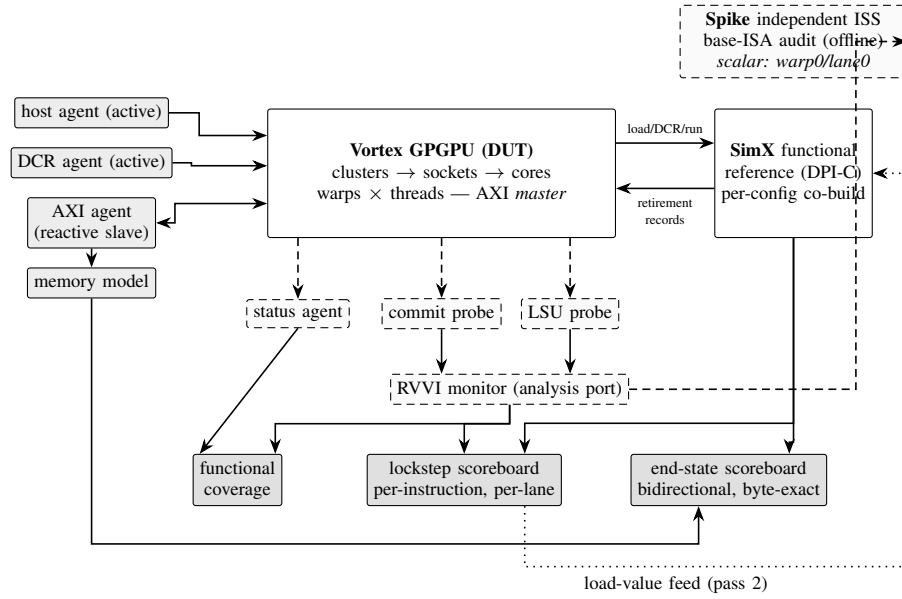


Fig. 1. The environment, in brief. A GPGPU executes programs rather than being driven by them, so the UVM roles invert: agents are reactive slaves and launchers around a bus-master DUT, stimulus is a compiled kernel, and two checkers—a byte-exact bidirectional end-state scoreboard and a per-instruction, per-lane lockstep comparator—render verdicts against the project’s own functional simulator (SimX), linked live over DPI.

### III. THE ENVIRONMENT IN ONE FIGURE

Figure 1 shows the shape of the environment. Three design choices matter for anyone porting the pattern.

**Roles invert around a GPU.** The DUT is the bus master; UVM agents are reactive—an AXI responder backed by a memory model services fetch and data traffic, host and device-configuration agents drive the launch protocol, and a passive layer of bound probes (commit arbiter, load-store writeback, scheduler, caches) feeds checkers and coverage without ever being a checker itself. One compiled environment runs any cluster/core/warp/thread topology from plusargs, with elaboration-time assertions cross-checking every testbench parameter against its DUT counterpart.

**Verdicts are a taxonomy, and “unverifiable” is a first-class member.** Every run ends PASS, FAIL, or UNVERIFIABLE—the last when the golden model itself cannot judge (say, the reference aborts on an instruction it cannot decode). Classifying rather than force-fitting such runs is what preserved our ability to find reference-model bugs later.

**Checkers must be proven able to fail.** Each verdict-rendering checker carries a permanent fault-injection guard in regression: flip one bit of one store (the end-state scoreboard must catch it), drop one store (the reverse pass must catch it), corrupt one retirement (the lockstep must name the exact instruction and lane). We once nearly accepted a gate whose negative tests reported PASS because their injection plusargs were absent; the codified rule since then is *verify the injection message, never the verdict*. Every number we quote below is backed by checkers that have been seen to go red.

### IV. LOCKSTEP FOR A WARP MACHINE: FIVE RULES

Per-instruction comparison against a functional model is standard for CPUs [6], [8]; a SIMT GPU breaks the standard in five specific ways. Each forced a rule, and each rule generalizes to any warp-based DUT:

- 1) **Retire order is not program order.** The commit arbiter retires whichever unit is ready; we watched an instruction retire after two later ones. *Align by the per-warp issue counter (uuid), sorted—never by cycle.*
- 2) **The reference has no uuid.** SimX leaves its identifier at zero, so there is no cross-key. *Align both streams by per-(core, warp) program-order position; the DUT’s uuid high bits supply the attribution.*
- 3) **One instruction is not one record.** A single load can commit in fragments with overlapping thread masks (0xd then 0xe) as lanes’ memory responses arrive. *Aggregate records by uuid with mask-union before comparing.*
- 4) **Load data is stale at the arbiter and unsafe to compare naively.** Loads of uninitialized memory legitimately differ between models—429 false mismatches on a known-good kernel. *Tap the load-store unit for true values, and compare only where memory state is provably shared; defer the rest to the end-state check.*
- 5) **Performance counters differ by construction.** *mcycle* and friends are timing artifacts. *Flag them volatile; compare address and destination, skip the data.*

With the rules in place, the comparator validated lane-exact—zero mismatches, zero orphans—across five configurations up to two clusters, two cores, eight warps, including

nested-divergence towers that exercise the mask-union rule through reconvergence. One honest exception class: the DUT’s `fsqrt.s` is one ULP off correctly-rounded IEEE 754, so its writeback is tolerated within one ULP (tallied, never silent), and a second-pass feed pushes the accepted result into the reference so the deviation cannot cascade. That is not a fudge; it is a declared, logged exception—and Section VI shows what being strict about exceptions buys you.

## V. RACY PROGRAMS, VERIFIED

Fenceless multi-core programs have no architecturally defined result: the two models legitimately resolve the same race differently, and a per-instruction compare drowns in false mismatches. The standard responses are to waive such programs or compare only final state. We do better with a two-pass *load-value feed*: pass one records the DUT’s loaded values at the divergent sites; pass two re-runs the reference *following* those values, so both models share each race outcome—and any remaining difference is a real defect. On the flagship case, twenty racy loads cascading into 138 divergent retirements collapse to a residual of **zero over 5,432 retirements**.

The soundness assumption is stated, not smuggled: the feed is honest exactly when a divergent load is a genuine race resolution rather than a load-data bug. In the flagship case we confirmed it by trace—the loaded word still held its pristine initialization value, proving a cross-core ordering race. Where the fixed point genuinely does not exist, we say so: asynchronous interrupts (the models take them at different instruction boundaries) and races that steer control flow remain unverified at instruction granularity, classified rather than forced green.

## VI. THE HEADLINE LESSON: AUDIT THE GOLDEN MODEL TOO

A lockstep flow compares two artifacts, and only one of them is the RTL. SimX is maintained by the same project as the DUT—and our per-instruction compare found it defective, more than once.

The reference fetched instructions at the exact byte program counter. A computed jump to an odd target—which the RTL tolerates by word-aligning—made SimX read across an instruction boundary and abort, silently discarding verification results as “unverifiable” runs. The lockstep’s own divergence report localized the bug. Tellingly, the obvious fix—masking the jump target in the model—made things *worse*: 21,968 phantom mismatches, because it corrected the model toward the specification instead of toward the DUT. A golden model’s job is to mirror the design as built; where design and specification disagree, that disagreement is a finding, not something to launder.

Then came the symmetric exchange with the fuzzing world. We ran FuzzGPU’s reported Vortex bugs at our pin. One reproduced as an RTL defect (an FPU writeback rule violation, firing the RTL’s own assertion). The second—publicly described as a hardware bug and fixed upstream as such—reproduced as a divergence in which *the hardware was right and the*

*golden model was wrong*: the upstream fix lives entirely in the simulator’s decoder. Only a DUT-versus-reference comparison could attribute that correctly. Two methodologies, two directions of mislabeling, both caught: their “hardware bug” was our golden-model bug, and our hardware’s ISA deviation (JALR not clearing the target LSB, with real architectural consequences) had been independently found by their fuzzing. If you run lockstep against a project-internal model, budget for verifying the model—it will need it.

## VII. COVERAGE YOU CAN DEFEND

Functional coverage for a GPU needs a layer that does not exist anywhere in the scalar world: SIMT microarchitecture coverage. Our model adds warp-divergence depth crossed with the reconvergence stack, scratchpad bank conflicts, and memory-coalescing classes—to our knowledge the first published such layer—atop a third-party ISA layer and a protocol layer. Closure on the primary configuration reached 98.1% of covergroup bins (94.7% total), with the ISA layer separately at 83.1% of bins. Full provenance, configuration by configuration, is in our preprint [13].

Three disciplines make those numbers defensible rather than decorative:

**Exclusions are machine-generated and gate-checked.** Every waiver cites `RTL file:line`; a blocking “hits-invariant” check aborts any merge in which a waived bin had actually been hit. That gate has caught three classes of defect in operation, including its own operator: an exclusion set generated with the wrong configuration’s keys aborted a merge that would otherwise have silently corrupted a 110-run bank. A methodology whose bookkeeping catches its operator’s mistakes is the minimum bar for trusting a signed-off percentage.

**Coverage definitions are verification targets too.** Our bank -conflict coverpoint, defined over *accepted* requests, could never hit—the crossbar admits one winner per bank per cycle. The “stimulus gap” was a definition defect; deliberate bank-hostile stimulus proved it by producing zero conflicts, and redefining the coverpoint over *offered* requests made it reachable and honest. Validate the coverage model against the RTL, not just the RTL against the model.

**Negative results are results.** Ninety additional random programs from a scalar generator closed *zero* coverage bins—every category bit-identical. Coverage saturates inside a stimulus kind; more seeds of the same kind buy nothing. Only SIMT-aware generated kernels closed the SIMT-specific targets (divergence depth, bank conflicts). Kind, not volume, moves a mature coverage model.

## VIII. WHAT DOES IT COST?

Measured on four kernels across three modes—plain environment, lockstep on, lockstep plus the two-pass feed—the answer is: far less than the value of what it catches (Table I). Lockstep adds no measurable wall-clock overhead at any kernel size; its real cost is memory, roughly 29% peak RSS at the largest working set for the second model instance.

TABLE I  
WHAT THE CHECKING STACK COSTS (MEDIAN; QUESTASIM).

| Kernel         | Mode     | Wall   | Peak RSS |
|----------------|----------|--------|----------|
| small baseline | plain    | 15.3 s | 296 MB   |
|                | lockstep | 15.0 s | 299 MB   |
|                | +feed    | 14.8 s | 299 MB   |
| large memory   | plain    | 851 s  | 549 MB   |
|                | lockstep | 871 s  | 708 MB   |
|                | +feed    | 856 s  | 708 MB   |

The feed is free. One accident was instructive: the FPU kernel under plain lockstep fails exactly as the `fsqrt` exception-class design predicts, and passes with the feed armed—the cheapest possible demonstration that the exception machinery does what its documentation says.

## IX. WHAT THE CHECKING FOUND

Beyond the reset relay and the golden-model defects: the JALR LSB deviation (independently corroborated by Fuz-zGPU, and later found by us to cascade into silent misaligned accesses through address-generation instructions); a watchdog constant written as `1**N`—one, for any `N`—that disabled stall detection at every scale, independently fixed upstream after we reported it; a fire-and-forget AXI write path with no error response handling, proven by injecting `SLVERR` on every seventh transaction (166 firings, no recovery); a `fsqrt.s` one ULP off correctly rounded, invisible to any end-state tolerance and caught only by per-instruction compare; and a memory-scheduler timeout denominated in picoseconds that floods L2/L3 runs with tens of thousands of false alarms. The full register—sixty-plus entries, each with evidence and a disposition—ships with the artifact [13].

## X. TAKEAWAYS

- **A silenced checker is a deaf checker.** Guards added to make bring-up survivable must be tracked and re-litigated; the assertion we restored had been right all along, and its message had been the only witness to a real defect.
- **Expect coverage to move *down* when you fix certain bugs.** If it does not, you may be counting coverage earned in illegitimate states.
- **Lockstep verifies two artifacts.** Treat the golden model as a verification target, or it will quietly misattribute findings—possibly at a security conference’s expense.
- **Gate your waivers.** An exclusion that cannot prove it excluded nothing real is a liability.
- **Stimulus kind beats stimulus volume.** Buy generators that target what your coverage model says is missing, not more seeds of what it already has.
- **The GPU is not a big CPU.** Inverted agent roles, split retirements, and divergence-aware alignment are the price of admission—and, once paid, everything else transfers.

## XI. CONCLUSION

Open hardware now includes processors complex enough to deserve industrial verification, and attackers have noticed. We built that verification for Vortex—lockstep, racy-program checking, and a SIMT coverage model—and the exercise taught the harder lesson: the discipline you impose on the design must be imposed on every artifact in the loop, including the reference model and the coverage model itself. The environment, the evidence registers, and every number’s provenance are open [13]. The boulder rolls; so does the regression.

## ACKNOWLEDGMENT

The authors thank the Vortex group at Georgia Tech and the maintainers of `core-v-verif`, `riscv-dv`, `RVVI`, `riscvISACOV`, and Spike.

## REFERENCES

- [1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics,” in *Proc. MICRO-54*, 2021, pp. 754–766.
- [2] Z. Gao *et al.*, “Fuzzing open-source GPU hardware with SIMT program generation,” in *Proc. 35th USENIX Security Symp.*, 2026, pp. 2465–2484.
- [3] OpenHW Group, “core-v-verif.” [Online]. Available: <https://github.com/openhwgroup/core-v-verif>
- [4] CHIPS Alliance, “riscv-dv.” [Online]. Available: <https://github.com/chipsalliance/riscv-dv>
- [5] OpenHW Group / Imperas, “RISC-V Verification Interface.” [Online]. Available: <https://github.com/riscv-verification/RVVI>
- [6] Imperas Software, “ImperasDV.” [Online]. Available: <https://imperas.com/imperasdv>
- [7] lowRISC, “OpenTitan.” [Online]. Available: <https://github.com/lowRISC/opentitan>
- [8] Y. Xu *et al.*, “Towards developing high-performance RISC-V processors using agile methodology,” in *Proc. MICRO-55*, 2022.
- [9] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson, “The design and implementation of a verification technique for GPU kernels,” *ACM TOPLAS*, vol. 37, no. 3, 2015.
- [10] G. Li *et al.*, “GKLEE: Concolic verification and test generation for GPUs,” in *Proc. PPoPP*, 2012, pp. 215–224.
- [11] L. Di Guglielmo, F. Fummi, and G. Pravadelli, “Vacuity analysis for property qualification by mutation of checkers,” in *Proc. DATE*, 2010, pp. 262–267.
- [12] T.-T. Tsai *et al.*, “NVBitFI: Dynamic fault injection methodology for GPU applications,” in *Proc. DSN*, 2021, pp. 64–76.
- [13] S. Moussa *et al.*, “SIMT-aware lockstep verification and functional-coverage closure methodology for an open-source RISC-V GPGPU: a UVM 1.2 environment,” arXiv preprint (cs.AR), 2026.
- [14] *IEEE Standard for UVM Language Reference Manual*, IEEE Std 1800.2-2020, 2020.
- [15] Y. A. Manerkar *et al.*, “RTLCheck: A property checking approach for custom memory consistency models,” in *Proc. MICRO-50*, 2017, pp. 713–725.